

Doc. no.: P0492R0
Date: 2016-11-26
Reply to: Beman Dawes <bdawes at acm dot org>
S. Davis Herring <herring at lanl dot gov>
Nicolai Josuttis <nico at josuttis dot de>
Jason Liu <jasonliu dot development at gmail dot com>
Billy O'Neal <bion at microsoft dot com>
P.J. Plauger <pjp at dinkumware dot com>
Jonathan Wakely <cxx at kayari dot org>

Audience: Library

Proposed Resolution of C++17 National Body Comments for Filesystems(R0)

This document proposes resolutions of C++17 CD National Body Comments from [P0488R0](#) related to Filesystems (27.10 [filesystems]). Some "Late" Comments from [P0489R0](#) are also considered.

The proposed resolutions in this paper represent the consensus of the Library Working Group's Filesystem small group (SG), unless otherwise indicated.

This paper is a snapshot of current work in progress. Proposed wording for some NB comments is still being drafted. Proposed wording in this snapshot is relative to the CD, [N4606](#). At the request of the Project Editor, all proposed wording in the next revision will be based on the post-Issaquah version of the C++ working paper.

NB Technical and general comments

- US 25: `has_filename()` is equivalent to just `!empty()`
- US 31: Everything is defined in terms of one implicit host system
- US 32: Meaning of 27.10.2.1 unclear
- US 33: Definition of *canonical path* problematic
- US 34: Are there attributes of a file that are not an aspect of the file system?
- US 35: What synchronization is required to avoid a file system race?
- US 36: Symbolic links themselves are attached to a directory via (hard) links
- US 37: The term “redundant current directory (dot) elements” is not defined
- US 40: Not all directories have a parent.
- US 43: Concerns about *encoded character types*
- US 44: Definition of path in terms of a string requires leaky abstraction
- US 45: Generic format portability compromised by unspecified root-name
- US 46: *filename* can be empty so productions for *relative-path* are redundant
- US 48: Multiple separators are often meaningful in a *root-name*
- US 49: What does “method of conversion method” mean?
- US 51: Failing to add / when appending empty string prevents useful apps
- US 52: `remove_filename()` postcondition is not by itself a definition
- US 53: `remove_filename()`'s name does not correspond to its behavior
- US 54: `remove_filename()` is broken
- US 55: `replace_extension()`'s use of path as parameter is inappropriate
- US 56: Remove `replace_extension()`'s conditional addition of period
- US 57: On Windows, absolute paths will sort in among relative paths
- US 58: `parent_path()` behavior for root paths is useless
- US 59: `filename()` returning path for single path components is bizarre
- US 60: `path("/foo/").filename()==path(". ")` is surprising
- US 61: Leading dots in `filename()` should not begin an extension
- US 62: It is important that `stem()+extension()==filename()`
- US 63: `lexically_normal()` inconsistently treats trailing "/" but not "/" as directory
- US 185: Fold `error_code` and `non-error_code` signatures into one signature
- FI 14: `directory_entry` comparisons are members
- US 73, CA 2: *root-name* is effectively implementation defined
- US 74, CA 3: The term “pathname” is ambiguous in some contexts
- US 75, CA 4: Extra flag in path constructors is needed

US 76, CA 5: *root-name* definition is over-specified.
US 77, CA 6: *operator/* and other appends not useful if arg has *root-name*
US 78, CA 7: Member `absolute()` in 27.10.4.1 is overspecified for non-POSIX-like O/S
US 79, CA 8: Some operation functions are overspecified for implementation-defined file types

NB comments submitted as editorial

US 38: Duplicates §17.3.16
US 39: Remove note: Dot and dot-dot are not directories
US 41: The term “parent directory” for a (non-directory) file is unusual
US 42: Pathname resolution does not always resolve a symlink
US 47: “.” and “..” already match the name production
US 50: 27.10.8.1 ¶ 1.4 largely redundant with ¶ 1.3

Late comments

To be supplied

Markup

SG internal notes to itself. Who does what may change to balance workloads. These will be gone by the final proposal revision.

SG informative notes that will remain in this proposal but do not become part of the WP.

Instructions to the project editor.

Text to be removed from the WP.

Text to be added to the WP.

NB Technical and general comments

Comments related to `filename()`

These comments are tightly coupled and best resolved as a whole.

US 25; 27.10.8.4.10 [path.query]: `has_filename()` is equivalent to just `!empty()`

Status: Open

No Proposed wording. The real issue is a possibly incorrect post-condition for `remove_filename()`.

SG recommends revisiting this comment after other filename related comments have been resolved.

US 37; 27.10.4.12 [fs.def.normal.form]: The term “redundant current directory (dot) elements” is not defined

Status: Accept with modifications

Change 27.10.4.12 [fs.def.normal.form]:

normal form

A path with no **redundant** current directory (*dot*) elements **except possibly one at the end**, no redundant parent directory (*dot-dot*) elements, and no redundant *directory-separators*. The normal form for an empty path is an empty path. The normal form for a path ending in a *directory-separator* that is not the root directory has a current directory (*dot*) element appended. A path in normal form is said to be *normalized*. The process of obtaining a normalized path from a path that is not in normal form is called *normalization*. [*Note*: The rule that appends a current directory (*dot*) element supports operating systems like OpenVMS that use different syntax for directory names and regular file names. —end note]

Beman comments: Since iteration, etc, no longer adds a dot, we need to either strike "The normal form for a path ending in a *directory-separator* that is not the root directory has a current directory (*dot*) element appended." or add a note or example explaining the rationale. But check the meeting notes; I have a vague recollection we already discussed this.

US 51; 27.10.8.4.3 [path.append] ¶2.3: Failing to add / when appending empty string prevents useful apps

Status: Reject

Resolved by [US-74/CA-3](#)

US 52; 27.10.8.4.5 [path.modifiers] ¶5: `remove_filename()` postcondition is not by itself a definition

Status: Accept with modifications

Also see LWG [LWG 2665](#).

Change 27.10.8.4.5 [path.modifiers] ¶5:

Postcondition: !has_filename().

Effects: If has_filename() then remove the minimum number of trailing characters from pathname such that there is one less element in [begin(), end()). Otherwise, no effect.

US 53; 27.10.8.4.5 [path.modifiers] ¶7: remove_filename()'s name does not correspond to its behavior

Status: Open

Because of a possible trailing "/", this comment is technically correct. The real question (for the full LWG) is do we want to make a name change at this late date for a name that is "good enough".

This issue probably needs to be resolved before C++17 ships, since a name change is a breaking change.

Bikeshed: remove_last_element() rather than the suggested remove_component().

Caution: "remove_filename" appears at least seven times in sub-clause 27.10. All will need to be changed if the name changes.

Caution: "replace_filename" presumably also would need to be renamed.

US 54; 27.10.8.4.5 [path.modifiers] ¶10: remove_filename() is broken

Status: Open

Need to resolve [US 53](#) first.

US 60; 27.10.8.4.9 [path.decompose] ¶6: path("/foo/").filename()==path(".") is surprising

Status: Open

The filename() behavior at issue is specified in 27.10.8.4.9 [path.decompose] ¶6:

Returns: empty() ? path() : *--end().

So the underlying problem is in 27.10.8.5 path iterators [path.itr] ¶4:

⁴ For the elements of pathname in the generic format, the forward traversal order is as follows:

(4.1) — The *root-name* element, if present.

(4.2) — The *root-directory* element, if present. [Note: the generic format is required to ensure lexicographical comparison works correctly. —end note]

(4.3) — Each successive *filename* element, if present.

(4.4) — *dot*, if one or more trailing non-root slash characters are present.

The ¶4.4 iterator behavior has always been surprising and controversial, so the LWG SG looked at the possible ways of dealing with trailing non-root slash characters:

dot, if one or more trailing non-root slash characters are present.

Not acceptable because it loses the fact that the path represents a directory, yet knowing that is required for some operating systems (e.g. POSIX, z/OS, OpenMVS). We ensure that reconstruction by iteration yields the same path as the original for some definition of "same".

dot slash, if one or more trailing non-root slash characters are present.

Not acceptable because a slash is semantically ambiguous; it is already used to indicate a root, so also using it to indicate a directory path would be a trap all too easy to fall into.

dot an empty element, if one or more trailing non-root slash characters are present.

This works, and was discussed at length by the SG. It appears to resolve a number of concerns raised by other NB comments and by the SG during discussion of those comments. A paper is needed to identify all the ripple effects of this change, and propose wording changes where needed or desirable.

LWG 2665 remove_filename() post condition is incorrect

Although not a NB comment, LWG 2665 needs to be coordinated with this set of comments, particularly US-52. It is mentioned here as a reminder to the LWG.

Other comments

US 31; 27.10 [filesystems]: Everything is defined in terms of one implicit host system

Status: Open

That's by design, aimed at achieving portable syntax and portable behavior. And that objective has been largely achieved in practical real-world use for many years. No proposed wording.

Waiting for a paper or issue with proposed wording. Without proposed wording, there is no consensus for change.

US 32; 27.10.2.1 [fs.conform.9945] ¶3: Meaning of POSIX conformance unclear

Status: Open

The comment requests "Clarify that ¶2 governs and an error must be reported in such cases".

Change 27.10.2.1 [fs.conform.9945] ¶2:

Implementations are not required to provide behavior that is not supported by a particular file system. [Example: The FAT file system used by some memory cards, camera memory, and floppy disks does not support hard links, symlinks, and many other features of more capable file systems, so implementations are not required to support those features on the FAT file system but instead are required to report an error as described above. —end example]

Jonathan Wakely will supply improved wording.

US 33; 27.10.4.2 [fs.def.canonical.path]: Definition of *canonical path* problematic

Status: Accept

The comment appears to be correct in that the definition of canonical path is not used in the WP.

Remove 27.10.4.2 [fs.def.canonical.path]:

canonical path

An absolute path that has no elements that are symbolic links, and no dot or dot-dot elements (27.10.8.1).

US 34; 27.10.4.5 [fs.def.filesystem]: Are there attributes of a file that are not an aspect of the file system?

Status: Accept with modifications

Change 27.10.4.5 [fs.def.filesystem]:

A collection of files and certain of their attributes.

US 35; 27.10.4.6 [fs.def.race]: What synchronization is required to avoid a file system race?

Status: Open

No proposed wording. 27.10.4 Terms and definitions [fs.definitions] is not the proper place to describe file system race behavior. 27.10.2.3 File system race behavior [fs.race.behavior] would be the proper place.

Waiting for a paper or issue with proposed wording. Without proposed wording, there is no consensus for change.

US 36; 27.10.4.9 [fs.def.link]: Symbolic links themselves are attached to a directory via (hard) links

Status: Accept with modifications

The SG provides wording, taken from POSIX definitions 3.130, Directory Entry (or Link).

Davis Herring will open an issue on a related concern.

Change 27.10.4.9 [fs.def.link]:

link

A directory entry that associates a filename with a file. A link is either a hard link (27.10.4.8) or a symbolic link (27.10.4.21). An object that associates a filename with a file. Several link entries can associate names with the same file.

US 40; 27.10.4.15 [fs.def.parent]: Not all directories have a parent

Status: Reject

No proposed wording. The definition in question is taken directly from the POSIX §3.268 *Parent Directory* definition.

US 43; 27.10.5 [fs.req] ¶4: Concerns about *encoded character types*

Status: Accept with modifications

No proposed wording.

The SG also detected a problem with conversions that fail, and a need that directory-separator detection occur at the code-unit level. Billy O'Neal and Jonathan Wakely will provide issue write ups.

Change 27.10.5 *Requirements* [fs.req] ¶4:

Note: Use of an encoded character type implies an associated character set and encoding. Since signed char and unsigned char have no implied character set and encoding, they are not included as permitted types. — end note]

US 44; 27.10.8 [class.path]: Definition of path in terms of a string requires leaky abstraction

Status: Accept with modifications

Proposed wording, including a note, has been applied to LWG 2734 to partially resolve US 44.

The portion of this comment not resolved by LWG 2734 is resolved by [US-74/CA-3](#)

Beman will check 2734, and coordinate with Marshall if needed, or add a comment to 2734

US 45; 27.10.8.1 [path.generic]: Generic format portability compromised by unspecified *root-name*

Status: Open

No proposed wording. Davis Herring is working on proposed wording.

US 46; 27.10.8.1 [path.generic]: *filename* can be empty so productions for *relative-path* are redundant

Status: Accept with modifications

Change [path.generic]:

```
relative-path:
  filename
  relative-path directory-separator
  relative-path directory-separator filename
  an empty path Note to editor: this line is deliberately not in code font
```

...

name:

A non-empty sequence of characters other than directory-separator characters.

SG needs to review Jonathan's wording above

US 48; 27.10.8.1 [path.generic] ¶1: Multiple separators are often meaningful in a *root-name*

Status: Accept with modifications

Change 27.10.8.1 [path.generic] ¶1:

Except in *root-name*, multiple successive *directory-separator* characters are considered to be the same as one *directory-separator* character.

US 49; 27.10.8.2.2 [path.type.cvt]: What does “method of conversion method” mean?

Status: Accept with modifications

Change 27.10.8.2.2 [path.type.cvt]:

- ¶1.2: method of conversion `method` is unspecified
- ¶1.3: method of conversion `method` is unspecified
- ¶1.4: method of conversion `method` is unspecified

Check the post-Issaquah working paper. This may have already been fixed as editorial.

US 55; 27.10.8.4.5 [path.modifiers] ¶11: `replace_extension()`'s use of `path` as parameter is inappropriate

Status: Accept with modifications

The SG believes `path` is the appropriate parameter type, but that the standard needs to clarify class `path` usage.

Change 27.10.8 [class.path] ¶1:

An object of class `path` represents a path (27.10.4.17) and contains a `pathname` (27.10.4.18). Such an object is concerned only with the lexical and syntactic aspects of a path. The path does not necessarily exist in external storage, and the `pathname` is not necessarily valid for the current operating system or for a particular file system.

[Note: Class `path` is used to abstract away the differences between the string types used by different operating systems to represent `pathnames`, and to perform conversions between encodings when necessary. — *end note*]

SG should review wording; "abstract away" was added during editing, rather than the previously agreed upon "support".

US 56; 27.10.8.4.5 [path.modifiers] ¶11.2: Remove `replace_extension()`'s conditional addition of period

Status: Reject

The SG presented such a proposal to the Library Evolution Working group in Issaquah, including a number of examples (See Nico's paper). There was no consensus for change. See the LEWG notes.

US 57; 27.10.8.4.8 [path.compare] ¶2: On Windows, absolute paths will sort in among relative paths

Status: Reject

Insufficient motivation for change. Billy O'Neal will open an issue for an unrelated concern discovered while processing this comment. See LWG nnnn.

US 58; 27.10.8.4.9 [path.decompose] ¶5: `parent_path()` behavior for root paths is useless

Status: Accept with modifications

Change [path.decompose]:

```
path parent_path() const;
```

-?- Let `parentEnd` be an instance of `path::iterator`, initialized as follows:

- if `has_relative_path()`, from `--end()`
- otherwise, from `end()`

-5- Returns: `(empty() || begin() == --end()) ? path() : pp`, where `pp` is a `path` constructed as if by starting with an empty path and successively applying operator `/=` for each element in the range `[begin(), --end())parentEnd)`.

US 59; 27.10.8.4.9 [path.decompose] ¶6: `filename()` returning path for single path components is bizarre

Status: Reject

`path` is filesystem's vocabulary type for both full paths and for path components, in the same sense that `std::string` might be used as the vocabulary type for both sentences and the words in the sentences. See US 55 for further discussion.

US 61; 27.10.8.4.9 [path.decompose] ¶8: Leading dots in `filename()` should not begin an extension

Status: Open

Beman needs to coordinate with Nico to apply LEWG guidance.

Cases that seem worth considering, with suggested decompositions:

<i>path p</i>	<i>p.stem()</i>	<i>p.extension()</i>
".bashrc"	".bashrc"	""
". .abc"	". .abc"	""
". . .abc"	". . .abc"	""
"abc . .def"	"abc"	". .def"
"abc . . .def"	"abc"	". . .def"

Change 27.10.8.4.9 [path.decompose] ¶ 8:

```
path stem() const;
```

Returns: if filename() contains a period but does not consist solely of one or two periods other than in a sequence of one or more beginning periods, returns the substring of filename() starting at its beginning and ending with the character before the last period first period of the last sequence of one or more periods. Otherwise, returns filename().

Change 27.10.8.4.9 [path.decompose] ¶ 10:

```
path extension() const;
```

Returns: if filename() contains a period but does not consist solely of one or two periods other than in a sequence of one or more beginning periods, returns the substring of filename() starting at the beginning of the rightmost sequence of periods and for the remainder of the path. Otherwise, returns an empty path object.

US 62; 27.10.8.4.9 [path.decompose] ¶11: It is important that stem()+extension()==filename()

Status: Open

Yes, it is important. But the note in ¶11 already says exactly that, so the question becomes "Should we make that part of the note normative?"

Saying that normatively would be a second way of saying what is already specified behavior, and saying the same normative thing twice is not good standardese. No proposed wording.

Review notes, and result of the LEWG vote.

US 63; 27.10.8.4.11 ¶1: lexically_normal() inconsistently treats trailing "/" but not "/" as directory

Status: Open

Need proposed wording for the *Proposed change* "Append the /. in all cases known to name directories (if it is in fact necessary)".

We can now resolve this in light of LEWG vote.

US 185; 27.10.7: Fold error_code and non-error_code signatures into one signature

Status: Reject

Does not handle one signature being noexcept(true) and the other noexcept(false), and many other problems. Even the submitter is no longer in favor. See LWG list thread "[filesystem] US-185 Eliminating dual signatures for operational functions", October 25, 2016.

FI 14; 27.10.12.3: directory_entry comparisons are members

Status: Reject

This is LWG issue 2761, which has been closed as NAD.

US 73, CA 2; 27.10.8.1 [path.generic]: root-name is effectively implementation defined

Status: Accept with modifications

Change [path.generic]:

root-name:

An operating system dependent name that identifies the starting location for absolute paths. If the operating system does not define at least one *root-name*, then the implementation defines a *root-name*. Implementations are permitted to define additional *root-names*. [*Note:* Many operating systems define a name beginning with two *directory-separator* characters as a *root-name* that identifies network or other resource locations. Some operating systems define a single letter followed by a colon as a drive specifier – a *root-name* identifying a specific device such as a disk drive. —end note]

Add a new paragraph at the end of [path.generic]:

If *root-name* is otherwise ambiguous, the possibility with the longest sequence of characters is chosen. [*Note:* on a POSIX-like operating system, it is impossible to have a *root-name* and a *relative-path* without an intervening *root-directory element*. -- end note]

US 74, CA 3; 27.10.8 [class.path]: The term “pathname” is ambiguous in some contexts

Status: Accept with modifications

SG needs to review the proposed wording for US-74/CA-3.

Discussion:

Many operations on `filesystem::path` are defined in terms of its exposition-only `pathname` member, which is defined implicitly by `path::native()` as the native format path. However, the native format need not be compatible with lexical operations like, for example, `path::root_name()` and `path::extension()`.

The proposed resolution below clarifies the distinct role played by the `+=` operator as mentioned in US 44 and addresses US 74/CA 3 (with a more extensive resolution).

See P0430R1 Section 2.1, for additional rationale. Beman needs to check with Jason to verify P0430R1 now just refers to US 74/CA 3 here for proposed wording.

Proposed resolution:

Remove In the [class.path] synopsis, remove the private (exposition-only) member from class path:

```
private:
    string_type pathname; // exposition only
```

Change [path.fmt.cvt]:

Option A; from Davis:

[*Note:* The format conversions described in this section are not applied on POSIX- or Windows-based operating systems because on these systems:

- The generic format is acceptable as a native path.
- There is no need to distinguish between native format and generic format in function arguments.
- Paths for regular files and paths for directories share the same syntax.

—end note]

Option B; from Beman:

[*Note:* The format conversions described in this section are usually not applied on POSIX- or Windows-based operating systems because on these systems:

- The generic format is acceptable as a native path.
- There is usually no need to distinguish between native format and generic format in function arguments.
- Paths for regular files and paths for directories share the same syntax.

Even on these operating systems, the format conversions may be applied for pathnames with implementation-defined *root-names* ([path.generic]).

—end note]

SG needs to choose between Option A or Option B.

Several functions are defined to accept *detected-format* character sequence arguments that represent paths using either the generic pathname format grammar ([path.generic]) or the native pathname format ([fs.def.native]). Such an argument is taken to be in the generic format if and only if it matches the generic format but is not acceptable to the operating system as a native path.

Function arguments that take character sequences representing paths may use the generic pathname format grammar (27.10.8.1) or the native pathname format (27.10.4.11). If and only if such arguments are in the generic format and the generic format is not acceptable to the operating system as a native path, conversion to native format shall be performed during the processing of the argument.

[Note: Some operating systems may have no unambiguous way to distinguish between native format and generic format arguments. This is by design as it simplifies use for operating systems that do not require disambiguation. An implementation for an operating system where disambiguation is required is permitted to distinguish between the formats. —end note]

Beman is concerned about the following paragraph; his markup of text from Davis may be in error:

- Would s/Define G(n) and N(g) as the functions/Let G and N be the implementation's functions/ be clearer?
- Are some of the '=' supposed to be '=='?

Pathnames are converted as needed between the generic and native formats in an operating-system-dependent manner. Define $G(n)$ and $N(g)$ as the functions that convert to the generic and native formats respectively. If $g=G(n)$ for some n , then $G(N(g))=g$; if $n=N(g)$ for some g , then $N(G(n))=n$. [Note: Neither G nor N need be invertible. —end note]

If the native format requires paths for regular files to be formatted differently from paths for directories, the path shall be treated as a directory path if its last element is a directory-separator, otherwise it shall be treated as a path to a regular file.

When a path is constructed from or is assigned a single representation separate from any path, the other representation is selected by the appropriate conversion function (G or N).

When the (new) value p of one representation of a path is derived from the representation of that or another path, a value q is chosen for the other representation. The value q converts to p (by G or N as appropriate) if any such value does so; q is otherwise unspecified. [Note: If q is the result of converting any path at all, it is the result of converting p . —end note]

[SG note: P0430R1 (for US 75/CA 4) adds support for explicitly specifying the format to use, at least in the constructor.]

Change [path.construct]:

```
path(const path& p);  
path(path&& p) noexcept;
```

Effects: Constructs an object of class `path` having the same native and generic format pathnames as `p` with `pathname` having the original value of `p.pathname`. In the second form, `p` is left in a valid but unspecified state.

```
path(string_type&& source);
```

Effects: Constructs an object of class `path` for which the pathname in the detected-format of `source` has `pathname` having the original value of `source` ([path.fmt.cvt]). `source` is left in a valid but unspecified state.

```
template <class Source>  
path(const Source& source);  
template <class InputIterator>  
path(InputIterator first, InputIterator last);
```

Effects: Let s be the effective range of `source` ([path.req]) or the range `[first, last)`, with the encoding converted if required ([path.cvt]). Finds the detected-format of s ([path.fmt.cvt]) and constructs an object of class `path` for which the pathname in that format is s . Constructs an object of class `path`, storing the effective range of `source` (27.10.8.3) or the range `[first, last)` in `pathname`, converting format and encoding if required (27.10.8.2).

```
template <class Source>  
path(const Source& source, const locale& loc);  
template <class InputIterator>  
path(InputIterator first, InputIterator last, const locale& loc);
```

Requires: The value type of `Source` and `InputIterator` is `char`.

Effects: Constructs an object of class `path`, storing the effective range of source or the range `[first, last)` in `pathname`, after converting format if required and Let `s` be the effective range of source or the range `[first, last)`, after converting the encoding as follows:

— If `value_type` is `wchar_t`, converts to the native wide encoding (27.10.4.10) using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`.

— Otherwise a conversion is performed using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`, and then a second conversion to the current narrow encoding.

Finds the detected-format of `s` (`[path.fmt.cvt]`) and constructs an object of class `path` for which the `pathname` in that format is `s`.

In the example in `[path.fmt.cvt]` ¶ 7.2, change "pathname" to normal font rather than code font.

Change `[path.assign]`:

```
path& operator=(const path& p);
```

Effects: If `*this` and `p` are the same object, has no effect. Otherwise, sets the native and generic format pathnames to those of `p` modifies `pathname` to have the original value of `p.pathname`.

Returns: `*this`.

```
path& operator=(path&& p) noexcept;
```

Effects: If `*this` and `p` are the same object, has no effect. Otherwise, sets the native and generic format pathnames to those of `p` modifies `pathname` to have the original value of `p.pathname`. `p` is left in a valid but unspecified state. [*Note:* A valid implementation is `swap(p)`. —end note]

Returns: `*this`.

```
path& operator=(string_type&& source);  
path& assign(string_type&& source);
```

Effects: Sets the native and generic format pathnames to those of `source` Modifies `pathname` to have the original value of `source`. `source` is left in a valid but unspecified state.

Returns: `*this`.

```
template <class Source>  
path& operator=(const Source& source);  
template <class Source>  
path& assign(const Source& source);  
template <class InputIterator>  
path& assign(InputIterator first, InputIterator last);
```

Effects: Let `s` be the effective range of source (`[path.req]`) or the range `[first, last)`, with the encoding converted if required (`[path.cvt]`). Finds the detected-format of `s` and sets the `pathname` in that format to `s` (`[path.fmt.cvt]`). Stores the effective range of source (27.10.8.3) or the range `[first, last)` in `pathname`, converting format and encoding if required (27.10.8.2).

Returns: `*this`.

Change `[path.append]` ¶2 & 3:

```
path& operator/=(const path& p);
```

Effects: If equivalent to `has_filename() || !has_root_directory() && is_absolute()`, then appends `path::preferred_separator` to the generic format `pathname`. [*Example:* `path("//host")/"foo"` and `path("//host/")/"foo"` both equal `"//host/foo"`. —end example]

Appends `path::preferred_separator` to `pathname` unless:

— an added *directory-separator* would be redundant, or

— an added *directory-separator* would change a relative path into an absolute path [*Note:* An empty path is relative. —end note], or

— `p.empty()` is true, or

— `*p.native().cbegin()` is a *directory-separator*.

Then appends `p.native()` to the native format `pathname` `pathname`.

Returns: `*this`.

[SG note: this proposed wording depends on LWG 2732 and the recently proposed definition of `has_filename()` and will change again to address LWG 2664 and US 77/CA 6. It already, simply by ignoring `p` entirely until such time, addresses US 51.]

Change [path.modifiers]:

```
path& make_preferred();
```

Effects: Each directory-separator is converted to preferred-separator in the generic format `pathname`.

[SG note: the existing definition of `remove_filename()` is broken; it is expected to be rewritten.]

Change [path.modifiers]:

```
path& replace_extension(const path& replacement = path());
```

Effects: `pathname` (the stored path) is modified as follows:

- Any existing `extension()` (27.10.8.4.9) is removed from the generic format `pathname` stored path, then
- If `replacement` is not empty and does not begin with a *dot* character, a *dot* character is appended to the generic format `pathname` stored path, then
- `operator+=(replacement)`; `replacement` is concatenated to the stored path.

Returns `*this`.

Change [path.modifiers]:

```
void swap(path& rhs) noexcept;
```

Effects: Swaps the contents (in all formats) of the two paths `pathname` and `rhs.pathname`.

Change [path.native.obs]:

```
const string_type& native() const noexcept;
```

Returns: The native format `pathname` `pathname`.

```
const value_type* c_str() const noexcept;
```

Returns: Equivalent to `native().c_str()` `pathname.c_str()`.

```
operator string_type() const;
```

Returns: `native()` `pathname`.

[Note: Conversion to `string_type` is provided so that an object of class `path` can be given as an argument to existing standard library file stream constructors and open functions. —end note]

```
template <class EcharT, class traits = char_traits<EcharT>,  
         class Allocator = allocator<EcharT>>  
basic_string<EcharT, traits, Allocator>  
string(const Allocator& a = Allocator()) const;
```

Returns: `native()` `pathname`.

Remarks: All memory allocation, including for the return value, shall be performed by `a`. Conversion, if any, is specified by 27.10.8.2.

Change [path.generic.obs]:

```
template <class EcharT, class traits = char_traits<EcharT>,  
         class Allocator = allocator<EcharT>>  
basic_string<EcharT, traits, Allocator>  
generic_string(const Allocator& a = Allocator()) const;
```

Returns: The generic format `pathname` `pathname`, reformatted according to the generic `pathname` format

(27.10.8.1).

Remarks: All memory allocation, including for the return value, shall be performed by a. Conversion, if any, is specified by 27.10.8.2.

```
std::string generic_string() const;
std::wstring generic_wstring() const;
std::string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

Returns: The generic format pathname `pathname`, reformatted according to the generic pathname format (27.10.8.1).

Remarks: Conversion, if any, is specified by 27.10.8.2. The encoding of the string returned by `generic_u8string()` is always UTF-8.

Change [path.decompose]:

```
path root_name() const;
```

Returns: `root-name`, if the generic format pathname `pathname` includes `root-name`, otherwise `path()`.

```
path root_directory() const;
```

Returns: `root-directory`, if the generic format pathname `pathname` includes `root-directory`, otherwise `path()`.

...

```
path relative_path() const;
```

Returns: A path composed from the generic format pathname `pathname`, if `!empty()`, beginning with the first *filename* after *root-path*. Otherwise, `path()`.

...

[SG note: the new definition of `filename()` needs to refer to the generic format.]

```
path stem() const;
```

Returns: Let `f` be the generic format pathname for `filename()`. Returns a path whose generic format pathname is

- `f`, if it contains no periods or consists solely of one or two periods;

- otherwise, the prefix of `f` ending before its last period.

[SG note: we intend to change this paragraph to not count the first character as a period.]

Returns: if `filename()` contains a period but does not consist solely of one or two periods, returns the substring of `filename()` starting at its beginning and ending with the character before the last period. Otherwise, returns `filename()`.

[Example:

```
std::cout << path("/foo/bar.txt").stem(); // outputs "bar"
path p = "foo.bar.baz.tar";
for (; !p.extension().empty(); p = p.stem())
std::cout << p.extension() << '\n';
// outputs: .tar
// .baz
// .bar
```

—end example]

```
path extension() const;
```

Returns: a path whose generic format pathname is the suffix of the generic format pathname for `filename()` not included in `stem()`. if `filename()` contains a period but does not consist solely of one or two periods, returns the substring of `filename()` starting at the rightmost period and for the remainder of the path. Otherwise, returns an empty path object.

Remarks: Implementations are permitted to define additional behavior for file systems which append

additional elements to extensions, such as alternate data streams or partitioned dataset names.

[*Example:*

```
std::cout << path("/foo/bar.txt").extension(); // outputs ".txt"
std::cout << path("/foo/bar").extension();    // outputs ""
```

—*end example*]

[*Note:* The period is included in the return value so that it is possible to distinguish between no extension and an empty extension. Also note that for a path `p`, `p.stem()+p.extension() == p.filename()`. —*end note*]

Change [path.query]:

```
bool empty() const noexcept;
```

Returns: true if the generic format pathname is empty, else false `pathname.empty()`.

...

```
bool is_absolute() const;
```

Returns: true if the native format pathname `pathname` contains an absolute path (27.10.4.1), else false.

[*Example:* `path("/").is_absolute()` is true for POSIX-based operating systems, and false for Windows-based operating systems. —*end example*]

Change [path.gen]:

```
path lexically_normal() const;
```

Returns: a path whose generic format pathname is the normal form ([fs.def.normal.form]) of the generic format pathname of `*this` `*this` in normal form ([fs.def.normal.form]).

Change [path.itr]:

Path iterators iterate over the elements of the generic format pathname `pathname` in the generic format (27.10.8.1).

A `path::iterator` is a constant iterator satisfying all the requirements of a bidirectional iterator (24.2.6) except that, for dereferenceable iterators `a` and `b` of type `path::iterator` with `a == b`, there is no requirement that `*a` and `*b` are bound to the same object. Its `value_type` is `path`.

Calling any non-const member function of a path object invalidates all iterators referring to elements of that object.

For the elements of the generic format pathname `pathname` in the generic format, the forward traversal order is as follows:

...

Change [fs.op.canonical]:

```
path canonical(const path& p, error_code& ec);
path canonical(const path& p, const path& base, error_code& ec);
```

Effects: Converts `p`, which must exist, to an absolute path that has no symbolic link, *dot*, or *dot-dot* elements in its generic format pathname `"."`, or `".."` elements.

...

Change [fs.op.current_path]:

```
path current_path();
path current_path(error_code& ec);
```

Returns: The absolute path of the current working directory, obtained in the native format as if by POSIX `getcwd()`. The signature with argument `ec` returns `path()` if an error occurs.

...

Rationale:

The lexical operation specifications leave some room for variation among implementations, but not as much as may be required for native formats; further relaxation would compromise the utility of the operations for portable, predictable

path manipulation. Instead, we specify a usable but flexible mapping between the two formats and define each operation in terms of one (or, occasionally, both).

The second bullet item in the first note in [path.fmt.cvt], which says for POSIX- or Windows-based operating systems "There is no need to distinguish between native format and generic format in function arguments.", may not be true on Windows even if the other two bullet items are true.

US 75, CA 4; 27.10.8.4.1 [path.construct]: Extra flag in path constructors is needed

Status: Accept with modifications

See P0430R1 Section 2.2 for wording.

US 76, CA 5; 27.10.8.1 [path.generic]: *root-name* definition is over-specified

Status: Accept with modifications

See P0430R1 Section 2.3.1 for wording.

US 77, CA 6; 27.10.8.4.3 [path.append]: *operator/* and other appends not useful if arg has *root-name*

Status: Accept with modifications

See P0430R1 Section 2.3.2 for wording.

US 78, CA 7; 27.10.15.1 [fs.op.absolute]: Member `absolute()` in 27.10.4.1 is overspecified for non-POSIX-like O/S

Status: Accept with modifications

See P0430R1 Section 2.4.1 for wording.

US 79, CA 8; Several: Some operation functions are overspecified for implementation-defined file types

Status: Accept with modifications

See P0430R1 Section 2.4.2 for wording.

NB comments submitted as editorial

There was concern that some of these might involve substantive changes, so the SG processed them as ordinary technical comments.

US 38; 27.10.4.13 [fs.def.symlink]: Duplicates §17.3.16

Status: Accept

Project editor will make change.

US 39; 27.10.4.15 [fs.def.parent]: Remove note: Dot and dot-dot are not directories

Status: Accept

Project editor will make change.

US 41; 27.10.4.16 [fs.def.parent.other]: The term “parent directory” for a (non-directory) file is unusual

Status: Reject

Insufficient motivation for change. "parent directory" is the term we use to describe the containing directory; we're (already) following POSIX there, even though the POSIX definition is a bit weird.

US 42; 27.10.4.21 [fs.def.symlink]: Pathname resolution does not always resolve a symlink

Status: Reject

There is an apparent corner case contradiction between the POSIX definition and other places in the POSIX standard. The [fs.def.symlink] definition is identical to the POSIX 3.381 Symbolic Link definition () and the SG prefers to keep it that way.

US 47; 27.10.8.1 [path.generic]: “.” and “..” already match the name production

Status: Accept with modifications

Proposed resolution: that this is not editorial, but we know the fix. **Davis Herring to supply updated proposed wording.**

US 50; 27.10.8.3 [path.req]: path requirements ¶ 1.4 largely redundant with ¶ 1.3

Status: Reject

Beman to open an issue to expose this concern to additional experts. This is not a priority issue for shipping C++17.

Late comments from P0489R0

Quoting P0489R0, "These comments were not submitted in time to be registered as National Body Comments, but should be considered as possible issues against SC 22, N3151, ISO/IEC CD 14882."

Late comments 15-47 apply to 27.10 [filesystems]. Except for as noted below, the submitter of these late comments has agreed to open LWG issues for any late comments that are still a concern.

Late 15; 27.10.8.4.11 ¶3.1: A “mismatched element” cannot be equal to an iterator

Late 16; 27.10.8.4.11 ¶3.3.1: `lexically_relative("..\\foo")` produces nonsense.

Late 17; 27.10.8.4.11 ¶3.3.2: Behavior not always appropriate

Late 18; 27.10.8.4.11 ¶4: Behavior wrong in case corresponding to comment Late17

Late 19; 27.10.8.5 ¶2: It is surprising that a path is a container of paths

Recommend NAD. See [US 55](#), [US 59](#).

Late 20; 27.10.11 ¶1: Anemic status structure

Late 21; 27.10.12: `directory_entry` is just a trivial wrapper for path

Late 22; 27.10.13 ¶6: `directory_iterator` cumbersome to assemble full path with iterator’s directory

Late 23; 27.10.14.1 ¶28: `disable_recursion_pending()` name is ugly implementation detail

Late 24; 27.10.15.1 ¶1: `absolute()` can produce nonsense result

Late 25; 27.10.15.4 ¶4.2: What attributes does `copy_file()` copy?

Late 26; 27.10.15.6 ¶5: `create_directories()` complexity assumes syscalls take constant time

Late 27; 27.10.15.7 ¶5: `create_directory()` specification prevents sensible security measures

Late 28; 27.10.15.10: `create_symlink()` might misbehave if `to` is a directory

Late 29; 27.10.15.13: Access to file ID is superior to `equivalent()`

Late 30; 27.10.15.13 ¶3: `equivalent()`'s `s1==s2` check is ill-formed and could race

Late 31; 27.10.15.13 ¶5: Surprising that `equivalent()`'s `error_code` overload can throw

Late 32; 27.10.15.13 ¶5 : `equivalent()` should not reject special files outright

Late 33; 27.10.15.21: `is_other()` result surprising

Late 34; 27.10.15.25 ¶5: Could `last_write_time()` guarantee the error direction

Recommend submitting a paper that researches whether this is in fact possible. If it is possible, providing proposed wording would be helpful.

Late 35; 27.10.15.26: Why is there no way to read permissions?

Recommend NAD. They are available in `file_status`. If anyone thinks a separate function would be useful, they should

submit an issue or a paper.

Late 36; 27.10.15.26: The `permissions()` `error_code` overload should be `noexcept`
Status: Accept

Change 27.10.15.26 [fs.op.permissions] and 27.10.6 Header <filesystem> synopsis [fs.filesystem.syn]:

```
void permissions(const path& p, perms prms, error_code& ec) noexcept;
```

Late 37; 27.10.15.26 ¶2: `permissions()` actions should be separate parameter

Matt Austern pointed out this issue and its fix for the TS, but it got dropped on the floor in the TS shipping rush and has never been fixed. We should probably fix it now or never. Beman will propose wording, and ask the LWG/LEWG give it priority 1 in Kona.

Late 38; 27.10.15.26 ¶2: Unimplementable implication that `permissions()` is atomic

Late 39; 27.10.15.29 ¶3: Suprising `relative()` behavior for symlinks

Late 40; 27.10.15.31 ¶1: `remove_all()` unclear for symlink/

Late 41; 27.10.15.31 ¶3: Would be useful for `remove_all()` to report successful removal count on error

Late 42; 27.10.15.33 ¶1: `resize_file()` *Postcondition* missing argument
Status: Accept with modifications

Check the post-Issaquah working paper. This may have already been fixed as editorial.

Change 27.10.15.33 [fs.op.resize_file]:

```
Postcondition: file_size(p) == new_size.
```

Late 43; 27.10.15.33 ¶3: Add POSIX `ftruncate()` equivalent function

Recommend opening issue. Such a feature might be useful in 27.9 [file.streams] for C++2n.

Late 44; 27.10.15.38: `system_complete()` name inconsistent with similar functions in subclause

Late 45; 27.10.15.38 ¶6: Remove `system_complete()` unless it can be made reliable

Late 46; 27.10.15.40 ¶2: Question about `weakly_canonical()` *Effects*

Late 47; 27.10.15.40 ¶4: `weakly_canonical()` suggested caching is incorrect
